Modelo documental de MongoDB

1. Alcance y convenciones

- Base de datos: `gestion_documental`.
- Colecciones: `users`, `folders`, `files`, `shares`, `activity_log`.
- GridFS: `fs.files` y `fs.chunks` (bucket `fs`).
- Todos los `_id` son `ObjectId`; las fechas son `Date` en UTC.
- `createdAt` y `updatedAt` los aporta Mongoose (`timestamps: true`). `activity_log` solo tiene `createdAt`.
- Los nombres de colección se fijan explícitamente para evitar el pluralizado automático.

Principios de diseño

1. **Referenciar** lo que crece sin límite o tiene vida propia: usuarios, carpetas, archivos, permisos e historial.
2. **Embeber** lo pequeño, acotado y que se lee junto: rol, credenciales, `storageUsed`, `storageQuota`, la cadena de ancestros de una carpeta y las copias de nombres en el historial.
3. **No duplicar la jerarquía en los archivos**: la carpeta guarda su cadena de ancestros y el archivo solo referencia su carpeta.
4. **Soft delete como estado**, no como colección aparte: se conservan `_id`, referencias y ubicación original.
5. **El historial sobrevive a todo**: guarda `resourceName` y `actorName` como copias.

Relaciones

Relación	Campo
users 1 — N folders	`ownerId`
users 1 — N files	`ownerId`
folders 1 — N folders	`parentId` (jerarquía)
folders 1 — N files	`folderId`
files 1 — 1 fs.files	`gridFsId`
users 1 — N shares	`ownerId` / `granteeId`
files o folders 1 — N shares	`resourceType` + `resourceId`
users 1 — N activity_log	`actorId`
files o folders 1 — N activity_log	`resourceType` + `resourceId` (referencia lógica)

2. Colección `users`

Identidad, credenciales, rol, estado y contabilidad de almacenamiento. No contiene arreglos de archivos ni de carpetas.

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador.
`email`	string	Único, en minúsculas. Se usa para el login.
`passwordHash`	string	Hash bcrypt. `select: false`; nunca se devuelve por la API.
`name`	string	Nombre visible.
`role`	string	`ADMIN` o `USER` (por defecto `USER`).
`isActive`	boolean	Por defecto `true`. Desactivar no borra historial ni recursos.
`storageUsed`	number	Bytes ocupados. Incluye la papelera; solo baja al purgar.
`storageQuota`	number	Cuota máxima en bytes (por defecto 52 428 800 = 50 MB). Nunca menor que `storageUsed`.
`lastLoginAt`	Date | null	Último login correcto.
`createdAt`, `updatedAt`	Date	Alta y última modificación.

Ejemplo:

```
{
  _id: ObjectId("66f1a1c2e4b0a1b2c3d4e5f6"),
  email: "ana.torres@serviciosprofesionales.com",
  passwordHash: "$2b$12$Q9m8V0kY...",
  name: "Ana Torres",
  role: "USER",
  isActive: true,
  storageUsed: 184320,
  storageQuota: 52428800,
  lastLoginAt: ISODate("2026-10-18T09:12:00Z")
}
```

Embebido vs. referenciado

Decisión	Justificación
Credenciales, rol, `storageUsed` y `storageQuota` embebidos	Son 1:1 con el usuario, pequeños y se leen en cada login y antes de cada subida; evita un `$lookup` en la ruta crítica.

Decisión	Justificación
Sin arreglos `fileIds` o `folderIds`	Crecen sin límite y obligarían a reescribir el usuario en cada subida.
`ownerId` referenciado en carpetas y archivos	El propietario es una entidad independiente; no se duplica nombre ni correo.
Usuario desactivado, no borrado	`isActive: false` conserva `_id`, historial y referencias.

Cuota de almacenamiento

- `available = max(storageQuota - storageUsed, 0)`.
- Al subir se exige `storageUsed + size <= storageQuota`; al reemplazar contenido, `storageUsed + (sizeNuevo - sizeAnterior) <= storageQuota`.
- La verificación ocurre **antes de escribir en GridFS**, por lo que un rechazo (413) no deja binarios huérfanos.
- Solo un `ADMIN` cambia la cuota, con un entero positivo no menor que `storageUsed`.
- Sin transacciones, dos subidas simultáneas podrían excederla por poco; la reconciliación lo corrige.

Índices

Índice	Tipo	Motivo
`{ email: 1 }`	Único	Login y prevención de duplicados.
`{ role: 1, isActive: 1, createdAt: -1 }`	Compuesto	Listado administrativo por rol y estado.

3. Colección `folders`

Jerarquía de carpetas con **`parentId`** + **`ancestors`** + **`depth`** (patrón *materialized path* con ObjectIds).

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador.
`name`	string	Nombre. Único entre hermanas activas.
`ownerId`	ObjectId	Referencia a `users`.
`parentId`	ObjectId | null	Carpeta padre; `null` = raíz.
`ancestors`	ObjectId[]	Cadena de raíz a padre, sin incluir la propia carpeta.
`depth`	number	`ancestors.length`. Raíz = 0.
`isDeleted`, `deletedAt`, `deletedBy`,	—	Estado de papelera (sección 6).

Campo	Tipo	Descripción
`deletedRootId`		
`createdAt`, `updatedAt`	Date	Automáticos.

Ejemplo de jerarquía Proyectos → 2026 → Informes:

```
{ name: "Proyectos", parentId: null, ancestors: [], depth: 0 }
{ name: "2026", parentId: a1, ancestors: [a1], depth: 1 }
{ name: "Informes", parentId: a2, ancestors: [a1, a2], depth: 2 }
```

Reglas de la jerarquía

1. **Invariante:** raíz con `ancestors = []` y `depth = 0`; hija con `ancestors = [...padre.ancestors, padre._id]` y `depth = padre.depth + 1`.
2. **Propiedad:** una subcarpeta pertenece al mismo `ownerId` que su padre; no se mueve a un árbol ajeno.
3. **Sin ciclos al mover:** para mover `F` bajo `P` se rechaza si `P._id == F._id` o si `P.ancestors` contiene `F._id`. Además `P` debe estar activa, ser del mismo dueño y respetar `MAX_DEPTH` (100).
4. **Mover un subárbol:** se actualizan `ancestors` y `depth` de todos los descendientes (`{ ancestors: F._id }`) con `updateMany` y una pipeline, sin recursión en JavaScript.
5. **Borrado en cascada:** los descendientes se localizan con `{ ancestors: F._id }`.
6. **Restauración:** solo si el padre está activo (409 en caso contrario) y el nombre no choca con una hermana activa (409).

Embebido vs. referenciado

Decisión	Justificación
`parentId` referenciado	Relación básica; permite validar el padre.
`ancestors` embebido	Cadena acotada por `MAX_DEPTH`; un solo índice multikey resuelve subárboles y detecta ciclos sin recorrer la base.
`depth` embebido	Derivado de `ancestors`; permite limitar profundidad.
Hijos no embebidos	Un arreglo de hijos crece sin límite y se reescribe en cada alta o movimiento.

Índices

Índice	Tipo	Motivo
`{ ownerId, parentId, name }` con filtro `isDeleted: false`	Único parcial	Nombres únicos entre hermanas activas, sin impedir reutilizar el nombre de una carpeta en papelera. Comparación exacta; no se usan collations.
`{ ancestors: 1, isDeleted: 1 }`	Multikey	Subárboles, cascadas y validación de ciclos.

Índice	Tipo	Motivo
`{ ownerId, isDeleted, updatedAt: -1 }`	Compuesto	Listados por propietario.
`{ ownerId, isDeleted, deletedAt: -1 }`	Compuesto	Papelera del usuario.
`{ deletedRootId: 1 }` con filtro `isDeleted: true`	Parcial	Restaurar o purgar una cascada.
`{ ownerId, name }`	Compuesto	Búsqueda por nombre.

4. Colección `files`

Metadatos de archivos `.txt` y `.md`. El contenido vive en GridFS y se referencia con `gridFsId`.

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador.
`name`	string	Nombre con extensión. Único en su carpeta activa.
`ownerId`	ObjectId	Propietario; coincide con el dueño de la carpeta.
`folderId`	ObjectId | null	Carpeta contenedora; `null` = raíz del usuario.
`extension`	string	`txt` o `md`.
`mimeType`	string	`text/plain` o `text/markdown`.
`size`	number	Bytes.
`gridFsId`	ObjectId	`_id` en `fs.files` (1:1).
`contentHash`	string	SHA-256 del contenido.
`isDeleted`, `deletedAt`, `deletedBy`, `deletedRootId`	—	Estado de papelera.
`createdAt`, `updatedAt`	Date	Automáticos.

Ejemplo:

```
{
  name: "informe-final.md",
  ownerId: ObjectId("66f1a1c2e4b0a1b2c3d4e5f6"),
  folderId: ObjectId("670000000000000000000a3"),
  extension: "md", mimeType: "text/markdown", size: 48213,
  gridFsId: ObjectId("6720000000000000000ab01"),
  isDeleted: false
}
```

Relación con GridFS

`files` es la fuente de verdad de los metadatos; `fs.files` guarda el binario con `filename` y `metadata.fileId`.

- `files.gridFsId` es único: un documento no apunta a dos binarios.
- **Subir:** validar extensión y contenido, escribir en GridFS y crear el documento en `files`.
- **Editar contenido:** escribir un nuevo binario sin `metadata.fileId`, actualizar `gridFsId`, `size`, `contentHash` y `updatedAt`, borrar el binario anterior y enlazar el nuevo. Así se respeta el índice único mientras coexisten.
- **Eliminar definitivamente:** se borran `fs.chunks`, `fs.files`, el documento y sus comparticiones.
- Al renombrar se actualiza también `filename` en GridFS.

Embebido vs. referenciado

Decisión	Justificación
Metadatos embebidos en `files`	Son pequeños y permiten listar y filtrar sin tocar GridFS.
Contenido referenciado en GridFS	Soporta streaming, evita el límite de 16 MB por documento y es un requisito explícito.
Sin arreglo de archivos dentro de `folders`	Crecimiento ilimitado y reescrituras.
Sin duplicar el texto en `files`	Evitaría desincronización con el binario.

Índices

Índice	Tipo	Motivo
`{ ownerId, folderId, name }` con filtro `isDeleted: false`	Único parcial	Nombre único por carpeta activa.
`{ folderId, isDeleted, name }`	Compuesto	Listar el contenido de una carpeta.
`{ ownerId, isDeleted, updatedAt: -1 }`	Compuesto	Archivos recientes.
`{ ownerId, isDeleted, deletedAt: -1 }`	Compuesto	Papelera.
`{ deletedRootId: 1 }` con filtro `isDeleted: true`	Parcial	Cascadas.
`{ gridFsId: 1 }`	Único	Relación 1:1 con GridFS.
`{ isDeleted, extension, createdAt: -1 }`	Compuesto	Reporte de archivos por tipo.
`{ isDeleted, createdAt: -1 }`	Compuesto	Reporte por rango de fechas.
`{ ownerId, isDeleted, extension,`	Compuesto	Filtros por dueño, tipo y fecha.

Índice	Tipo	Motivo
`createdAt: -1 }`		
`{ ownerId, name }`	Compuesto	Búsqueda por nombre.

5. Colección `shares`

Permisos explícitos de un archivo o carpeta hacia un usuario: relación muchos a muchos.

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador.
`resourceType`	string	`FILE` o `FOLDER`.
`resourceId`	ObjectId	Archivo o carpeta (referencia polimórfica).
`ownerId`	ObjectId	Dueño del recurso al conceder.
`granteeId`	ObjectId	Destinatario; distinto del dueño.
`access`	string	`READ`, `COMMENT` o `EDIT`.
`createdAt`, `updatedAt`	Date	Alta y último cambio de nivel.

Semántica de permisos

Jerarquía: `READ < COMMENT < EDIT`; el propietario es `OWNER` sin necesidad de un `share`.

- `READ`: ver metadatos, listar y descargar.
- `COMMENT`: lo de `READ`; reservado para comentarios futuros.
- `EDIT`: lo anterior, más renombrar y reemplazar contenido.
- Crear, mover, eliminar y compartir son exclusivos del propietario.
- El permiso de una carpeta aplica a todo su contenido actual y futuro, resuelto por herencia y no copiado.
- Permiso efectivo de un archivo: el máximo entre su permiso directo y los de su carpeta y de todas las carpetas en `folder.ancestors`. Para una carpeta, los suyos y los de sus ancestros.
- Un recurso en papelera devuelve `NONE`; al restaurarlo, el permiso vuelve a aplicar.
- Revocar el permiso de una carpeta elimina la herencia, pero no los permisos directos sobre archivos concretos.
- Sin compartición transitiva: un destinatario no puede re-compartir.
- `ADMIN` no tiene acceso implícito al contenido.
- Sin permiso suficiente el servidor responde 404, para no revelar que el recurso existe.

Por qué una colección separada

Alternativa	Problema
Arreglo `sharedWith` en `files` y `folders`	Crecimiento sin límite, reescrituras y consultas

Alternativa	Problema
	difíciles de "compartido conmigo".
Un documento por permiso (elegida)	Consultas en ambas direcciones, revocación simple e índice único contra duplicados.
Copiar permisos a cada archivo	Duplicación masiva y revocación incompleta.

Índices

Índice	Tipo	Motivo
`{ resourceType, resourceId, granteeId }`	Único	Un solo permiso por recurso y usuario.
`{ granteeId, resourceType, createdAt: -1 }`	Compuesto	"Compartido conmigo".
`{ ownerId, resourceType, createdAt: -1 }`	Compuesto	"Compartido por mí" y reportes.

6. Papelera y soft delete

La papelera no es una colección: se representa con cuatro campos en `folders` y `files`.

Campo	Descripción
`isDeleted`	`false` activo, `true` en papelera.
`deletedAt`	Fecha del borrado.
`deletedBy`	Usuario que borró.
`deletedRootId`	`_id` del recurso raíz de la cascada.

Ventajas: los `_id` y las referencias siguen siendo válidos, la ubicación original se conserva y restaurar es solo un cambio de estado.

Semántica de `deletedRootId`

- Archivo borrado suelto: `deletedRootId = archivo._id`.
- Carpeta `F`: `F` y cada descendiente **que no estaba ya en papelera** reciben `deletedRootId = F._id`. Los que ya estaban borrados conservan su raíz y su fecha, para que al restaurar `F` no se resuciten borrados anteriores.
- Restaurar `F`: se restauran los documentos con `deletedRootId = F._id`.
- Purgar `F`: se elimina todo su subárbol físicamente.

Reglas

1. Todo listado, búsqueda y descarga filtra `isDeleted: false`.
2. **Restaurar** exige padre activo y nombre libre (409 en caso contrario) y no modifica `storageUsed`.

3. **Purga definitiva:** se escribe primero el log, luego se borran binarios, comparticiones y documentos, y solo entonces se resta `storageUsed` con `$inc`.
4. Cada usuario ve su propia papelera; un recurso borrado deja de estar disponible para los invitados.
5. No hay expiración automática; la purga es explícita y nunca toca `activity_log`.

7. Colección `activity_log`

Auditoría inmutable: quién hizo qué, sobre qué recurso y cuándo. Nunca se actualiza ni se borra.

Campo	Tipo	Descripción
`actorId`	ObjectId | null	Usuario que ejecutó la acción; `null` para procesos del sistema o seed.
`actorName`, `actorRole`	string	Copias del nombre y rol en ese momento.
`action`	string	Acción del catálogo.
`resourceType`	string	`USER`, `FOLDER`, `FILE`, `SHARE`, `AUTH` o `SYSTEM`.
`resourceId`	ObjectId | null	Recurso afectado.
`resourceName`	string	Copia del nombre del recurso; mantiene la auditoría legible tras una purga.
`metadata`	object	Datos acotados (tamaño, permiso, contadores, `batchId`, etc.).
`createdAt`	Date	Marca de tiempo UTC. No hay `updatedAt`.

Catálogo de acciones: `AUTH_LOGIN`; `USER_CREATE`, `USER_UPDATE`, `USER_DEACTIVATE`, `USER_ACTIVATE`; `FOLDER_CREATE`, `FOLDER_RENAME`, `FOLDER_MOVE`, `FOLDER_DELETE`, `FOLDER_RESTORE`, `FOLDER_PURGE`; `FILE_UPLOAD`, `FILE_DOWNLOAD`, `FILE_RENAME`, `FILE_MOVE`, `FILE_UPDATE`, `FILE_DELETE`, `FILE_RESTORE`, `FILE_PURGE`; `TRASH_EMPTY`; `SHARE_GRANT`, `SHARE_UPDATE`, `SHARE_REVOKE`.

Ejemplo:

```
{
  actorId: ObjectId("66f1a1c2e4b0a1b2c3d4e5f6"), actorName: "Ana Torres", actorRole:
"USER",
  action: "FILE_UPLOAD", resourceType: "FILE",
  resourceId: ObjectId("671000000000000000000f1"), resourceName: "informe-final.md",
  metadata: { extension: "md", size: 48213 },
  createdAt: ISODate("2026-10-01T12:00:00Z")
}
```

Reglas de escritura

- Solo inserción: no existen endpoints de actualización ni de borrado.
- Los nombres se copian en el momento del evento; no se reconstruyen con `$lookup`.
- Las cascadas registran un evento raíz con contadores y un `batchId`, nunca arreglos sin límite.

- En las purgas, el evento se escribe **antes** de borrar el recurso.

Embebido vs. referenciado

Decisión	Justificación
`metadata` embebido	Cada acción tiene un esquema distinto y pequeño; evolucionable sin migraciones.
`actorId` y `resourceId` referenciados	Permiten correlacionar y reportar por id.
`actorName` y `resourceName` copiados	Auditoría válida aunque el usuario se desactive o el recurso se purgue.
Log no embebido en usuarios ni recursos	Crecería sin límite y desaparecería al purgar.

Índices

Índice	Motivo
`{ actorId, createdAt: -1 }`	Historial de un usuario en un rango.
`{ resourceType, resourceId, createdAt: -1 }`	Línea de tiempo y archivos con más operaciones.
`{ action, createdAt: -1 }`	Operaciones por tipo y por período.
`{ createdAt: -1 }`	Últimos eventos y rangos sin actor.
`{ "metadata.batchId": 1 }` (disperso)	Reconstruir una cascada.

8. GridFS

Colección	Contenido	Relación
`fs.files`	Un documento por binario: `length`, `chunkSize`, `uploadDate`, `filename`, `metadata`.	`files.gridFsId` → `fs.files._id` (1:1)
`fs.chunks`	Trozos binarios: `files_id`, `n`, `data`.	`fs.chunks.files_id` → `fs.files._id` (1:N)

Índices: `{ filename, uploadDate }` y `{ files_id, n }` único (por defecto del driver) y `{ "metadata.fileId": 1 }` único y disperso (creado por el seed). El contenido no se duplica en `files`; los reportes de tamaño usan `files.size`.

9. Resumen: embebido vs. referenciado

Información	Dónde vive	Decisión
Credenciales, rol, `storageUsed`, `storageQuota`	`users`	Embebido (1:1, ruta crítica)

Información	Dónde vive	Decisión
Jerarquía de carpetas	`parentId` + `ancestors`	Referencia y cadena embebida
Metadatos de archivo	`files`	Embebido
Contenido del archivo	`fs.files` y `fs.chunks`	Referenciado por `gridFsId`
Permisos	`shares`	Colección separada
Papelera	Campos de estado en `folders` y `files`	Embebido como estado
Historial	`activity_log`	Colección separada, solo inserción
Nombres históricos	`actorName`, `resourceName`	Copia embebida
Hijos de una carpeta, archivos de un usuario	—	No embebidos

10. Invariantes e integridad

1. `folder.ownerId` coincide con el de su padre.
2. `file.ownerId` coincide con el de su carpeta.
3. `folder.ancestors` no contiene a la propia carpeta y `depth == ancestors.length`.
4. El padre debe existir y estar activo para crear, mover o restaurar.
5. Nombres únicos entre hermanos activos (carpetas y archivos).
6. `shares.ownerId` coincide con el dueño real del recurso y `granteeId` es distinto de `ownerId`.
7. Un recurso con `isDeleted: true` no es accesible aunque tenga `shares`.
8. `files.gridFsId` es único y apunta a un `fs.files` existente.
9. `storageUsed` es la suma de `files.size` de los archivos no purgados del usuario (incluye papelera); se incrementa al subir y se decrementa solo al purgar.
10. `storageQuota` es un entero positivo, nunca menor que `storageUsed`; toda subida que lo exceda se rechaza con 413 antes de escribir en GridFS.
11. `activity_log` no se actualiza ni se elimina.
12. Los usuarios no se borran: se desactivan.

Orden seguro de operaciones (sin transacciones)

MongoDB se despliega como instancia independiente, por lo que **no se usan transacciones**. Cada operación se divide en pasos ordenados para que un fallo deje un estado recuperable, y las escrituras son idempotentes y reintentables. En operaciones **destructivas** el log va primero; en las aditivas o reversibles, al final.

Operación	Orden de pasos
Subir archivo	1) validar extensión, UTF-8 y cuota; 2) escribir en GridFS; 3) crear documento en `files`; 4) `$inc` de `storageUsed`; 5) log `FILE_UPLOAD`.

Operación	Orden de pasos
	Un fallo entre 2 y 3 deja un binario huérfano; entre 3 y 4, un `storageUsed` subestimado.
Editar contenido	1) validar y cuota con la diferencia; 2) escribir el nuevo binario; 3) actualizar `files`; 4) ajustar `storageUsed`; 5) borrar el binario anterior y enlazar el nuevo; 6) log `FILE_UPDATE`. Nunca se borra el binario viejo antes de actualizar la referencia.
Mover carpeta	1) validar destino y calcular `ancestors` y `depth`; 2) actualizar la carpeta; 3) actualizar descendientes por el prefijo antiguo; 4) log `FOLDER_MOVE`. Si falla entre 2 y 3, la reconciliación recalcula desde `parentId`.
Borrar a papelera	1) resolver la cascada por `ancestors`; 2) marcar `isDeleted`, `deletedAt`, `deletedBy`, `deletedRootId`; 3) log.
Restaurar	1) verificar padre activo y nombre libre; 2) limpiar los campos de papelera en la raíz y su cascada; 3) log.
Purga definitiva	1) **log** `FILE_PURGE` o `FOLDER_PURGE` con contadores y `batchId`; 2) borrar `fs.chunks` y `fs.files`; 3) borrar `shares`; 4) borrar documentos; 5) `$inc` negativo de `storageUsed` con el tamaño capturado en el paso 1. Los pasos 2 a 4 son idempotentes.

Nunca se acepta un recurso purgado sin evento de auditoría. Un job de reconciliación puede recalcular `storageUsed`, eliminar binarios huérfanos, detectar `gridFsId` sin binario y recalcular `ancestors` y `depth` desde `parentId`.

11. Reportes y colección utilizada

Todos se resuelven con **aggregation pipelines**; los de recursos vivos filtran `isDeleted: false` al inicio. Un ADMIN ve datos globales y un USER solo los suyos.

#	Reporte	Colección	Pipeline
1	Archivos por tipo	`files`	`$match`, `$group` por `extension` con `$sum` de conteo y tamaño.
2	Archivos en un rango de fechas	`files`	`$match` por `createdAt` (`$gte`, `$lte`), orden descendente.
3	Archivos con más operaciones	`activity_log`	`$match` de `resourceType: FILE`, `$group` por `resourceId`, `$sort`, `$limit`; nombre desde `resourceName`.
4	Top 10 usuarios con más archivos	`files`	`$group` por `ownerId`, `$sort`, `$limit: 10`, `$lookup` a `users`.

#	Reporte	Colección	Pipeline
5	Usuarios con más recursos compartidos	`shares`	`$group` por `ownerId`, `$sort`, `$lookup` a `users`.
6	Archivos compartidos con más usuarios	`shares`	`$match` de `FILE`, `$group` por `resourceId` con `$addToSet` de `granteeId`, `$lookup` a `files`.
7	Recursos compartidos por un usuario	`shares`	`$match` por `ownerId`, `$lookup` polimórfico a `files` y `folders`.
8	Recursos recibidos por un usuario	`shares`	`$match` por `granteeId`, `$lookup` polimórfico.
9	Archivos que nunca han sido compartidos	`files`, `shares`, `folders`	Archivos vivos sin share directo ni heredado de una carpeta compartida (`folder.ancestors`).
10	Carpetas con más archivos	`files`, `folders`	`$group` de archivos vivos por `folderId`, `$sort`, `$limit: 10`, `$lookup` a `folders`.
11	Historial de un usuario en un rango	`activity_log`	`$match` por `actorId` y `createdAt`, orden descendente.
12	Operaciones por tipo	`activity_log`	`$group` por `action`, `$sort`.
13	Operaciones por período	`activity_log`	`$group` con `$dateTrunc` (día o mes) sobre `createdAt`.
14	Principales acciones	`activity_log`	`$group` por `action` y `resourceType`, `$sort`, `$limit: 10`.

Notas: los reportes de actividad no filtran por recursos existentes y usan los nombres copiados; los conteos de compartición se basan en permisos explícitos, y la herencia se calcula con `folders.ancestors` solo cuando el reporte lo requiere.

Anexo A. Mapeo con los esquemas Mongoose

Elemento	Nombre físico	Clase	Archivo
Base de datos	`gestion_documental`	—	`MONGODB_URI`
Usuarios	`users`	`User`	`server/src/users/schemas/user.schema.ts`
Carpetas	`folders`	`Folder`	`server/src/folders/schemas/folder.schema.ts`
Archivos	`files`	`File`	`server/src/files/schemas/file.schema.ts`
Comparticiones	`shares`	`Share`	`server/src/shares/schemas/share.schema.ts`

Elemento	Nombre físico	Clase	Archivo
Historial	`activity_log`	`ActivityLog`	`server/src/activity-log/schemas/activity-log.schema.ts`
Binarios	`fs.files`, `fs.chunks`	Sin esquema Mongoose	`GridFSBucket` del driver

Todos los esquemas usan `timestamps: true` y `versionKey: false` (sin campo `__v`); `ActivityLog` usa `{ createdAt: true, updatedAt: false }`.

Tipos y referencias

Dato	Declaración Mongoose
Referencia	`@Prop({ type: SchemaTypes.ObjectId, ref: 'User' })`
Referencia opcional	`Types.ObjectId | null` con `default: null`
Enumeración	`enum` de TypeScript con `enum: Object.values(...)`
Arreglo de ids	`ancestors: [{ type: SchemaTypes.ObjectId, ref: 'Folder' }]`
Objeto libre	`metadata: SchemaTypes.Mixed`

Referencias declaradas: `Folder.ownerId`, `File.ownerId`, `Share.ownerId`, `Share.granteeId`, `ActivityLog.actorId` → `User`; `Folder.parentId`, `File.folderId` → `Folder`. `Share.resourceId` y `ActivityLog.resourceId` son referencias polimórficas sin `ref` fijo; su integridad se valida en los servicios.

Sincronización

`modelo.md` describe el diseño y los archivos `*.schema.ts` son la fuente ejecutable. Los índices se crean al registrar cada esquema, salvo los de GridFS, que crea el script de seed.